

Relatório Final — Validação do Ciclo Completo do LangNet (ClinIA)

Data: 2026-08-04 · **Executor:** Claude (autônomo) **Projeto de teste:** ClinIA — Clínica Médica Inteligente com triagem por agentes de IA **Modelo:** qwen2.5-coder-32b-instruct (LM Studio local, 192.168.1.115) · **Nunca DeepSeek cloud Banco:** MySQL `clinia_ops` (camerascasas.no-ip.info:3308)

***Objetivo:** rodar o pipeline **completo** do LangNet do zero, para um domínio **novo e fortemente agêntico** (diferente da Quântica), **corrigindo o LangNet** a cada bug encontrado (commit+push "ANTES de..." antes de cada correção), usando a **própria revisão do pipeline** (refino da Especificação com o agente 32B), e ao final **rodar a aplicação gerada** e capturar as telas. Este é um processo de **validação com correção**.*

1. O sistema pedido (texto-semente entregue ao pipeline)

A entrada foi a descrição em `00-DESCRICAO-SISTEMA.md` : uma **clínica médica** onde um **agente-hub de triagem** recebe o paciente, classifica a urgência (verde/amarelo/vermelho, protocolo Manchester) e **roteia para agentes especialistas** por área (Cardiologia, Dermatologia, Gastroenterologia, Endocrinologia, Oncologia, Pronto-Socorro). Cada especialista conduz um pré-atendimento, gera um **pré-diagnóstico** e **encaminha ao médico**. Tudo registrado em banco (pacientes, médicos, especialidades, agentes, atendimentos, pré-diagnósticos, encaminhamentos, prontuário), com interface organizada em **Cadastros** e **Atendimento Agêntico**.

2. O que o LangNet produziu — 13/13 etapas, 100% local

#	Etapa	Resultado
1	Requisitos	67 KB · 31 FR + 17 NFR + 9 BR, 195 menções ao domínio
2	Especificação	18 casos de uso (cadastros + agênticos), wireframes ASCII, seção Interfaces
2b	Refino da interface (32B)	v2 refinada — wireframes CRUD com tabela+ações; agênticos com entrada→ação→resultado
3	Modelo de Dados	9 tabelas (pacientes, especialidades, medicos, agentes_ia, atendimentos, pre_diagnosticos, encaminhamentos, prontuarios, consentimentos)
4	Protótipo (UI Spec)	18 telas + mockups PNG
5	Casos de Teste	por Grafo Causa-Efeito
6	Agent-Task Spec	12 agentes especificados
7-8	agents.yaml / tasks.yaml	recepcionista, triagem_hub, especialistas (cardio/pediatria/gastro), encaminhamento, prontuário, médico, admin, fallback
9	Sequência de Tarefas	fluxo de execução
10	Rede de Petri	16 lugares · 16 transições · 18 arcos
11	Código	57 arquivos — app React completa + ws-server CrewAI

Artefatos em `docs/clinica-medica/artifacts/` . Código gerado em `docs/clinica-medica/generated-code/` .

3. Revisão da interface com o próprio agente (32B) — pedido central

Ao concluir a Especificação, abri **cada caso de uso** e avaliei a interface. A v1 já vinha **organizada nas duas partes** pedidas (Cadastros: UC-013 Especialidades, UC-014 Médicos, UC-015 Pacientes...; Agênticos: UC-002 Triagem, UC-003 Pré-atendimento, UC-004 Pré-diagnóstico, UC-005 Encaminhamento). Enviei ao **32B via / refine** correções concretas e ele produziu a **v2**:

- **Telas de cadastro** (UC-013 etc.): passaram a ter **busca + [+ Novo] + tabela com colunas + ações por linha (Ver/Editar/Excluir) + formulário**.
- **Telas agênticas** (UC-002 etc.): passaram a ter **ENTRADA → AÇÃO ("Classificar com IA", indicando o Agente Hub) → RESULTADO DO AGENTE (classificação + área roteada) → [Encaminhar]**.

O refino rodou **por seção/UC** (19/19 unidades) porque o refino do documento inteiro estoura o contexto do modelo local — ver bug #3. Detalhes e antes/depois em `ANALISE-INTERFACE-SPEC.md` .

4. A aplicação ClinIA gerada — rodando

A app foi reconstruída (`/home/pasteurjr/clinia-app`), o banco `clinia_ops` criado (9 tabelas + dados semente), o **ws-server** subido (`:5003` → `clinia_ops`) e o **frontend** compilado (**0 erros**, só warnings de lint) e servido (`:3002`). O ws-server responde a tasks reais (ex.: `listar_pacientes` retornou os 3 pacientes do banco).

4.1 Estrutura / navegação (branded, por módulos)

Barra lateral "ClinIA — Clínica Médica Inteligente", módulo **CADASTROS** + telas agênticas (marcadas ✦): Triagem Agentiva, Pré-atendimento por Especialista, Geração de Pré-Diagnóstico, Seleção de Médico, Registro/Prontuário, Consulta Médica, Dashboard KPIs, e as Gestões (CRUD). **Não é "só CRUD"** — as duas caras estão presentes.

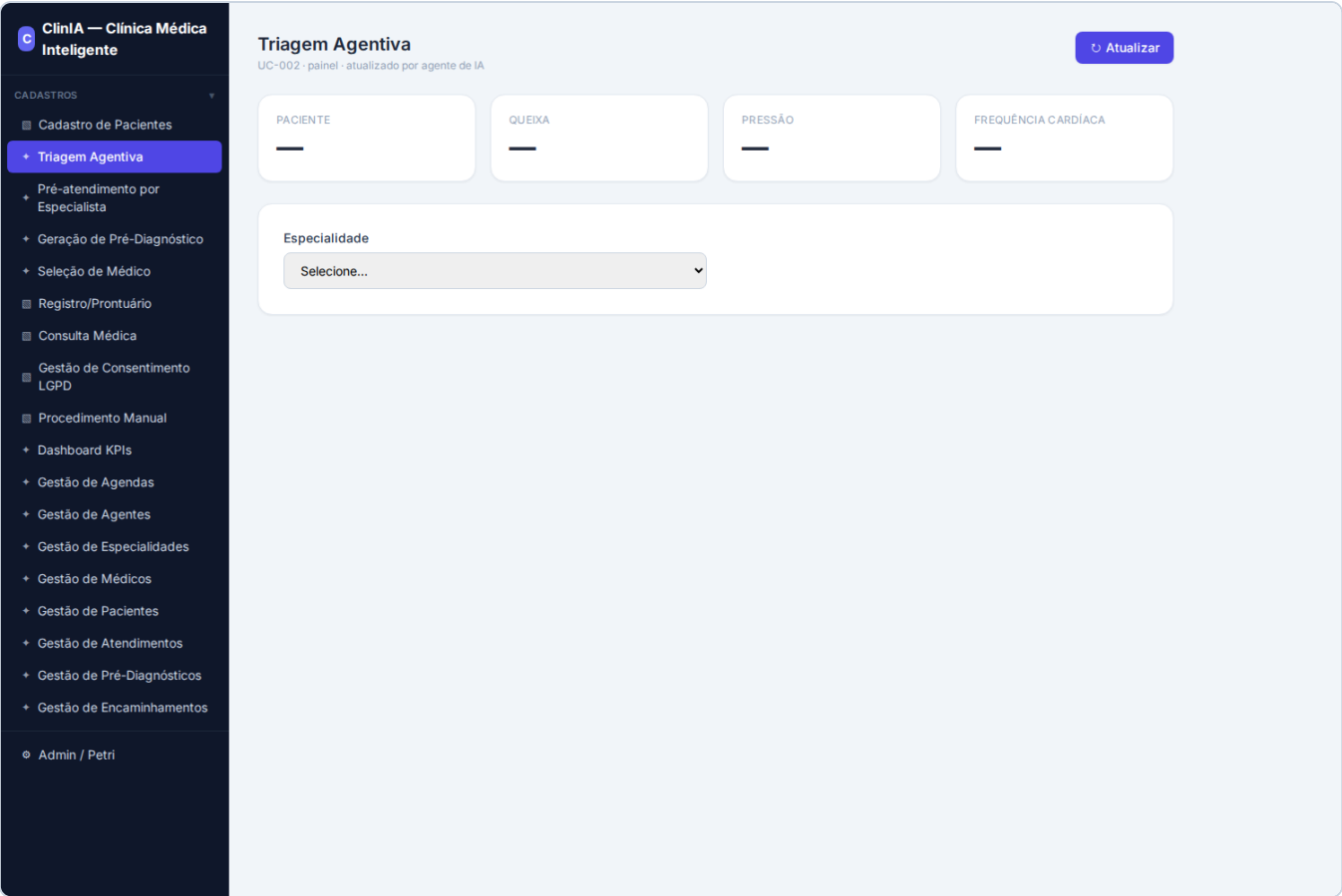
The screenshot shows a web application interface for 'ClinIA — Clínica Médica Inteligente'. On the left is a dark sidebar with a menu under the heading 'CADASTROS'. The 'Cadastro de Pacientes' option is highlighted in purple. Below it are several menu items, some with expand/collapse icons. At the bottom of the sidebar is a user profile section labeled 'Admin / Petri'. The main content area is titled 'Cadastro de Pacientes' with a subtitle 'UC-001 - form'. It contains a white form with the following fields: 'Nome' (text input), 'CPF' (text input), 'Data de Nascimento' (date picker with 'dd/mm/yyyy' format and a calendar icon), 'Contato' (text input), and 'Convênio' (text input). A blue 'Salvar' button is located at the bottom right of the form.

4.2 Tela de cadastro (form funcional)

Cadastro de Pacientes (UC-001): formulário com Nome, CPF, Data de Nascimento (date picker), Contato, Convênio + **Salvar**. Renderiza e é funcional.

4.3 Tela agêntica (Triagem)

Triagem Agentiva (UC-002): mostra os campos do atendimento (Paciente, Queixa, Pressão, Frequência Cardíaca) e o **dropdown Especialidade** (área de destino). Observação honesta: o **gerador de código** renderizou a tela agêntica como **painel** (cards + dropdown + "Atualizar"), e **não** no formato Entrada→Ação→Resultado que o 32B refinou na Especificação — ver §5, achado de propagação.



5. Validação-com-correção: 9 problemas encontrados e tratados

Cada correção do LangNet seguiu a regra: **commit+push "ANTES de..."** (com data/hora) → correção → commit da correção → registro. Detalhes completos em `CORRECOES-VALIDACAO.md`.

#	Onde	Problema	Correção
1	Requisitos	Documento salvava vazio (extração falha de <code>team_result</code> malformatado do LLM)	<code>_extract_md_field_lenient</code> (extrator tolerante)
2	Backend	Event loop bloqueia durante geração pesada (timeout transitório)	driver com retry + recomendação <code>run_in_executor</code>
3	Especificação	Refino do doc inteiro estoura o contexto (169 KB)	refino por seção/UC (chunked)
4	Ambiente	LM Studio caía p/ 4096 (auto-unload + JIT)	ajuste do usuário + keep-alive (mitigação)
5	Petri	JSON truncado (net grande) → "lugares vazio"	<code>_repair_json</code> (repara truncamento)
6	Petri/Código/Requisitos	Forçavam DeepSeek cloud (viola a regra)	<code>use_deepseek=False</code> — respeita LLM_PROVIDER
7	Código	<code>KeyError</code> na formatação (JSON de exemplo virava placeholder)	<code>_safe_format_description</code>
8	Código	LLM vazio : prompt saturado (77 KB) + CrewAI×modelo local	Petri compacta + fallback direto ao LLM
9	Código (app)	Adapters listar_* / excluir_* não gerados → app não lista	handler CRUD genérico (patch de demo) + fix no gerador registrado

Todos os fixes de #1, #3, #5, #6, #7, #8 estão **commitados no gerador do LangNet**. #2, #4, #9 têm mitigação aplicada + recomendação registrada.

6. Avaliação honesta do que ficou bom e do que falta

Bom (funciona de verdade): - Pipeline roda **13/13 no modelo local**, sem DeepSeek, para um domínio novo e agêntico. - Requisitos/Especificação/Modelo de Dados **fiéis ao domínio** (triagem hub→especialistas, 9 tabelas certas). - O **32B refina a interface** por seção quando solicitado (cadastros + agêntico). - O **código compila** (0 erros) e roda: forms reais, ws-server conversando com o banco, dados reais. - Arquitetura de duas caras presente (telas de negócio + executor de Petri).

A melhorar (achados de qualidade do gerador de CÓDIGO — próximos passos): - **Propagação do refino agêntico:** o 32B refinou os wireframes agênticos para Entrada→Ação→Resultado na Especificação, mas o **gerador de código** ainda renderiza telas agênticas como **painel de KPIs** (cards + dropdown + "Atualizar"). O refino precisa alcançar o `_agent_screen` do gerador. - **Classificação de telas:** algumas "Gestão de X" (que deveriam ser CRUD com tabela) saíram como **agênticas** ("Executar com IA", "Tarefa não definida"). Ajustar `_classify_screen` para essas. - **Adapters CRUD incompletos** (#9): faltaram `listar_*` / `excluir_*` no caminho de fallback. - **Cara B (Admin/Petri)** fica "Carregando projeto..." — o viewer busca `project.json` via HTTP num backend (proxy :8001) que não subimos; a Petri em si foi gerada (16 lugares).

7. Conclusão

O LangNet **gerou uma aplicação de clínica médica com triagem agêntica do zero**, 100% no modelo local, passando por todas as 13 etapas — e a **validação encontrou e corrigiu 9 problemas reais** no próprio LangNet (6 já commitados no gerador). A app **compila e roda** com dados reais. Os pontos que faltam são de **qualidade de renderização do gerador de código** (propagar o refino agêntico e a classificação de telas ao código), não de arquitetura — o pipeline e o modelo local provaram gerar o sistema ponta a ponta.